

ILP-Based Scheduling for High-Level Synthesis

Milestone 2: Scientific Contextualization and Tradeoff Analysis

Name	MD Rafiul Islam
Student ID	1232714
Course area	SS2026 ELE HW/SW Codesign Seminar
Topic	ILP-Based Scheduling for High-Level Synthesis
Document type	Structured context and tradeoff analysis

Purpose: This document is a structured milestone artifact for understanding, analysis, evaluation, and scientific communication. It is written as technical notes, not as copied final prose.

1. Milestone 2 goal

Milestone 2 places ILP-based scheduling into the broader scientific and engineering landscape. The central question is: where does ILP scheduling sit among other HLS scheduling strategies, and what tradeoffs does it create?

- Related approaches are not collected only for quantity; they are used to understand design alternatives.
- The comparison focuses on optimality, runtime, model complexity, hardware constraints, and deployability.
- The output should help justify why ILP is useful and why it is not always the practical default.

2. Context: HLS design flow

Scheduling is one decision inside a larger HLS flow. The surrounding tasks are allocation and binding. Allocation decides how many resources exist. Scheduling decides when operations execute. Binding maps scheduled operations to concrete resource instances.

Behavioral description -> operation graph -> allocation -> scheduling -> binding -> RTL datapath + c

This context matters because ILP scheduling cannot be judged only as a mathematical model. Its value is measured by the hardware it helps create: latency, functional unit count, register usage, multiplexers, and controller behavior.

3. Neighboring scheduling approaches

Approach	Main idea	Strength	Main weakness
ASAP / ALAP	Compute earliest and latest feasible control steps.	Good for mobility analysis and simple bounds.	Not enough alone for resource-constrained optimization.
List scheduling	Schedule ready operations using priorities.	Fast and practical for larger graphs.	May miss the optimal schedule.
Force-directed scheduling	Use distribution graphs to balance resource pressure.	Good area-time balancing heuristic.	More complex than list scheduling and still heuristic.
ILP scheduling	Use binary variables and linear constraints.	Can prove optimality for the formulated objective.	Solver/model size can become large.
Metaheuristics	Search using simulated annealing, genetic algorithms, tabu search, etc.	Can explore hard design spaces.	No simple optimality guarantee and parameter tuning is needed.

4. Tradeoff analysis

Tradeoff axis	ILP side	Alternative side
Optimality vs runtime	ILP can return an optimal schedule for the model.	Heuristics are faster but may be suboptimal.
Mathematical clarity vs implementation effort	Variables and constraints are explicit.	A heuristic can be easier to implement without a solver.
Small kernels vs full designs	Best for small/medium kernels or exploration.	Large designs often need heuristics or decomposition.
Offline vs online	Scheduling is normally offline at design time.	Runtime scheduling is uncommon for HLS operation graphs.
Resource cost vs latency	Can encode resource limits and latency objectives directly.	Heuristics may need special priority functions.

Context conclusion: ILP is not simply "better" than heuristics. It is stronger when exactness and clear constraints matter; it is weaker when model size and solving time dominate.

5. Consequences for embedded systems and FPGAs

- For FPGA accelerators, reducing latency may require more DSP blocks or LUT resources.
- For embedded systems, exact scheduling can help when a small arithmetic kernel is timing-critical.
- For area-sensitive designs, the designer may intentionally restrict resources and accept more cycles.
- For large data-flow graphs, the ILP model may be too heavy, so practical tools may mix exact and heuristic methods.
- The final RTL behavior should be checked because a valid schedule must still map cleanly to registers, datapath operations, and a controller.

6. Scope choices for the seminar paper

The seminar paper uses a compact arithmetic example instead of a large industrial benchmark. This is a defensible scope decision because the topic must also include mathematical basics, formal model, algorithm explanation, C/C++ feature implementation, an application example, and RTL realization.

- Included: binary decision variable idea, dependency constraints, resource constraints, latency objective, C++ search, VHDL/Quarta validation.
- Excluded or only briefly mentioned: loops, memory access conflicts, pipelining, branching, multi-cycle functional units, commercial solver integration.
- Reason for exclusion: these extensions would make the implementation and explanation too broad for a 5-page seminar paper.

7. Source-use notes

Reference	How it supports this topic
Teich and Haubelt, 2007	German HLS / HW/SW synthesis background: allocation, scheduling, binding, optimization.
De Micheli, 1994	Main textbook-level foundation for synthesis and optimization of digital circuits.
McFarland, Parker, Camposano, 1990	Classic overview of high-level synthesis and behavioral synthesis flow.
Paulin and Knight, 1989	Neighboring heuristic approach: force-directed scheduling for behavioral synthesis.
Cong et al., 2011	Modern FPGA-oriented HLS context and deployment relevance.
Rettberg, 2026 seminar slides	Course requirements: scientific paper, C/C++ feature, application example, RTL component realization.